

Lecture 19: TCP Part 1 — Connections, Sequencing, and Flow Control

EC 441 — Introduction to Computer Networking

Boston University, Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

1. Identify and explain the purpose of each field in a TCP segment header.
2. Explain the byte-stream model and why TCP sequence numbers count bytes rather than segments.
3. Trace a TCP three-way handshake, explaining the role of SYN, SYN-ACK, and ACK and why Initial Sequence Numbers are randomized.
4. Describe TCP connection teardown: the FIN/ACK exchange, half-close, RST, and why `TIME_WAIT` exists.
5. Explain how TCP achieves reliable delivery using cumulative ACKs and retransmission timers.
6. Explain TCP's RTT estimator as a discrete-time low-pass filter (EWMA), and explain why variance tracking (RTTVAR) is necessary for setting a robust RTO.
7. Explain fast retransmit: why three duplicate ACKs are a reliable signal of loss.
8. Describe flow control: how the receive window (rwnd) prevents the sender from overrunning the receiver's buffer.

1 From RDT Building Blocks to TCP

Lecture 18 built the reliable data transfer toolkit from first principles: sequence numbers, cumulative acknowledgments, retransmission timers, and pipelined protocols (Go-Back-N and Selective Repeat). We closed with a design question: *if you were building a reliable, full-duplex, byte-stream protocol on top of IP, what would you choose?*

TCP is the deployed answer. Every design decision it made connects directly to the building blocks from Lecture 18. Table 1 maps each L18 concept to TCP's choice.

This lecture covers the TCP segment header, connection establishment and teardown, reliable delivery mechanics (RTT estimation, fast retransmit), and flow control. Congestion control is deferred to Lecture 20.

Table 1: From L18 RDT building blocks to TCP’s design decisions

L18 concept	TCP’s answer
Sequence numbers	Byte offsets in the stream (not packet counts)
Cumulative ACK (GBN)	Default; SACK option adds SR-style selective acknowledgment
Receiver buffer (SR)	Required; advertised as the receive window (rwnd)
Pipelining	Effective window = $\min(\text{cwnd}, \text{rwnd})$
Single timer vs. per-packet	One RTO per connection, plus fast retransmit as an early signal
Full-duplex	Two independent byte streams, one in each direction

2 The TCP Segment Header

2.1 Header Format

A TCP segment consists of a header (minimum 20 bytes, up to 60 bytes with options) followed by the data payload. Figure 1 shows the header layout using the RFC bit-field convention.

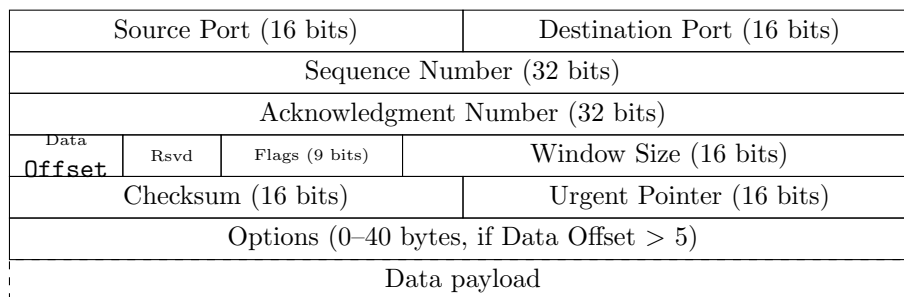


Figure 1: TCP segment header layout. Minimum header size is 20 bytes (Data Offset = 5).

2.2 Field-by-Field Walkthrough

Source Port / Destination Port (16 bits each). Identify the sending and receiving processes. Together with the source and destination IP addresses from the enclosing IP header, these four values form the *5-tuple* (proto, src IP, src port, dst IP, dst port) that uniquely identifies a TCP connection. All five fields are required for TCP demultiplexing; two connections from different clients to the same server port are distinguished because their source IP or source port differs.

Sequence Number (32 bits). The byte offset in the sender’s stream of the *first byte of data in this segment*. If the SYN flag is set (connection establishment), the sequence number is the Initial Sequence Number (ISN) and no data bytes are consumed — but the SYN itself logically consumes one sequence number so it can be retransmitted and acknowledged.

Acknowledgment Number (32 bits). The sequence number of the *next byte the sender of this segment expects to receive*. ACK = N means “I have received all bytes through $N - 1$; please send me byte N next.” Only meaningful when the ACK flag is set. On an established TCP connection the ACK flag is nearly always set; acknowledgment piggybacks on every data segment.

Data Offset (4 bits). Length of the TCP header in 32-bit words. Minimum value is 5 (= 20 bytes). This field tells the receiver where the data payload begins, analogous to IPv4’s IHL field.

Flags (9 bits). Table 2 lists each flag and its role.

Table 2: TCP flag bits

Flag	Name	Meaning
SYN	Synchronize	Initiate a connection; carries the ISN
ACK	Acknowledgment	Acknowledgment number field is valid
FIN	Finish	Sender has no more data; begin graceful close
RST	Reset	Abort the connection immediately
PSH	Push	Pass buffered data to the application without waiting
URG	Urgent	Urgent pointer field is valid (rarely used)
CWR	Congestion Window Reduced	Sender reduced cwnd in response to ECN (L20)
ECE	ECN-Echo	Received a congestion signal from the network (L20)

Window Size (16 bits). The *receive window* (rwnd): the number of bytes the sender of this segment is willing to accept beyond its last acknowledged byte. This is the primary flow-control mechanism. Without the Window Scale option the window is limited to 65535 bytes; on a 1 Gb/s link with 50 ms RTT the bandwidth–delay product alone is ~6 MB, making Window Scale essential.

Checksum (16 bits). Computed over a pseudo-header (source IP, destination IP, protocol = 6, TCP length) concatenated with the TCP header and data. Mandatory in both IPv4 and IPv6, unlike UDP’s optional checksum.¹

Urgent Pointer (16 bits). Used only when the URG flag is set; points to the last byte of out-of-band “urgent” data relative to the sequence number. Largely unused in modern applications.

Options (0–40 bytes). TCP options are negotiated during the handshake. Important ones:

- **MSS (Maximum Segment Size):** Each side advertises the largest segment it can accept. On an Ethernet network: MTU = 1500 bytes, minus 20 bytes IP header, minus 20 bytes TCP header, yields MSS = 1460 bytes.² The sender uses the *minimum* of its own and the remote MSS.
- **Window Scale:** Multiplies the 16-bit window field by 2^n (for negotiated shift n , 0–14), allowing receive windows up to ~1 GB. Essential for high-bandwidth-delay-product paths.
- **SACK (Selective ACK):** Adds Selective Repeat-style acknowledgment. The receiver reports specific byte ranges it has received out of order; the sender retransmits only the missing ranges rather than everything after the gap.
- **Timestamps:** Each segment carries a send timestamp; the ACK echoes it back, allowing the sender to measure RTT precisely for any acknowledged segment. Also used to disambiguate old sequence numbers on very high-speed links. Removes the need for Karn’s algorithm (Section 5.4).

2.3 The Byte-Stream Model

TCP treats the data flowing in each direction as a single continuous **byte stream**. There are no message boundaries at the TCP level. If an application sends two 100-byte writes, the receiver may

¹Wikipedia contributors. “Transmission Control Protocol.” https://en.wikipedia.org/wiki/Transmission_Control_Protocol

²Wikipedia contributors. “Maximum segment size.” https://en.wikipedia.org/wiki/Maximum_segment_size

read them as one 200-byte block, as four 50-byte reads, or any other combination. Applications that need message framing must impose it themselves: HTTP uses a blank line to terminate headers; TLS records carry explicit length fields.

Sequence numbers reflect this directly: they are *byte offsets*, not segment numbers. If the ISN is 1000 and the first segment carries 500 bytes, the sequence number of the second segment is 1500. An ACK of 1500 means “I have received all bytes through 1499; send me byte 1500 next.”

This contrasts with Go-Back-N and Selective Repeat from Lecture 18, which numbered *packets*. The effect is identical — the receiver can identify exactly what it has and what is missing — but byte-level granularity aligns naturally with the stream abstraction that applications use.

3 Connection Establishment: The Three-Way Handshake

3.1 Why a Handshake?

UDP needs no setup: send a datagram and it either arrives or it does not. TCP is different. Before any data can flow, both sides must:

1. Confirm that a connection exists and that both endpoints are reachable.
2. Exchange Initial Sequence Numbers so each side can track the other’s byte stream.
3. Negotiate options: MSS, Window Scale, SACK support, Timestamps.

The minimum number of messages needed to accomplish this reliably — so that both sides can confirm the other received the last message — is three. Hence the **three-way handshake**.

3.2 The Exchange

Figure 2 shows the message sequence. Each step is described below.

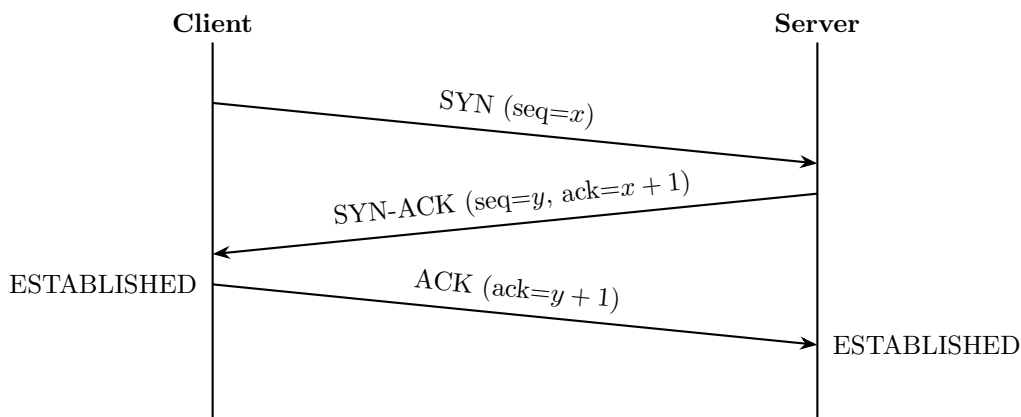


Figure 2: TCP three-way handshake. Data may flow after the third message.

1. **SYN.** The client sends a segment with $\text{SYN} = 1$ and its ISN x . No data payload. The SYN logically consumes one sequence number (the next byte of data will be $x + 1$), which makes it retransmittable: if the SYN is lost, the client retransmits it and the server can detect the duplicate via the sequence number.

2. **SYN-ACK.** The server responds with $\text{SYN} = 1$ and $\text{ACK} = 1$. The SYN carries the server's ISN y ; the ACK field $(x + 1)$ acknowledges the client's SYN. After sending this segment, the server knows both ISNs.
3. **ACK.** The client acknowledges the server's SYN with $\text{ack} = y + 1$. After this segment arrives, both sides have confirmed receipt of both ISNs and the connection is fully established.

3.3 ISN Randomization

Initial Sequence Numbers are not zero. Each side chooses a pseudo-random ISN, typically derived from a counter that advances at a fixed rate (RFC 6528 specifies a hash-based scheme involving the 5-tuple and a secret key). The reasons are:

1. **Stale segment rejection.** A TCP connection is identified by its 5-tuple. If a new connection reuses the same 5-tuple as a recently closed one and starts at $\text{ISN} = 0$, a segment delayed in the network from the old connection could appear valid in the new one. A pseudo-random ISN makes this collision astronomically unlikely.
2. **Security.** Predictable ISNs enabled TCP sequence-number prediction attacks: an off-path attacker who could guess the ISN could inject forged data into an established connection. Randomization defeats this class of attack.

Demo

```
# Show established TCP connections with process names
ss -t -p

# Open a raw TCP connection to a web server (port 80)
nc -v bu.edu 80

# In a second terminal while nc is running:
ss -t -p          # observe ESTABLISHED state

# Type the following, press Enter twice, then observe FIN teardown:
# GET / HTTP/1.0
```

4 Connection Teardown

4.1 Graceful Close: The FIN/ACK Exchange

TCP teardown uses a **four-way exchange** because each direction of the connection closes independently. Figure 3 shows the sequence.

1. **FIN.** The client sends $\text{FIN} = 1$. “I have no more data to send.” The FIN consumes one sequence number, making it reliably deliverable.
2. **ACK.** The server acknowledges the FIN. The client-to-server data direction is now closed.
3. **Half-close.** The server may still have data to send. It continues transmitting until its own application closes the connection.

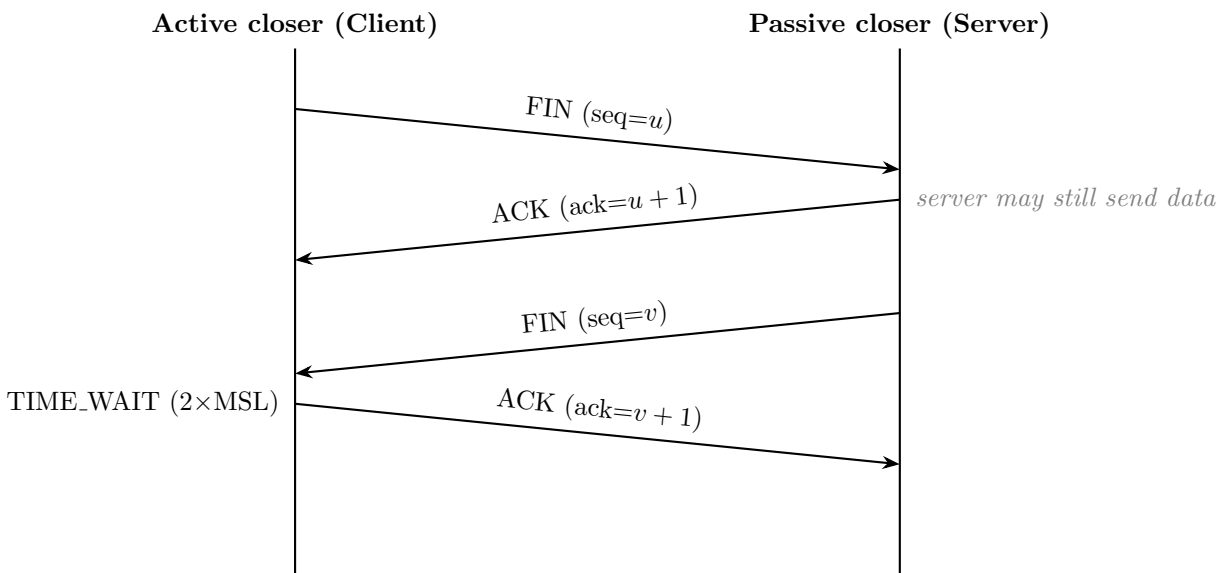


Figure 3: TCP connection teardown. The active closer enters **TIME_WAIT** after sending the final ACK.

4. **Server FIN.** When the server is done, it sends its own FIN. In practice, if the server has no data to send after the client's FIN, its ACK and FIN are often combined into a single segment, reducing the exchange to three messages.
5. **Final ACK.** The client acknowledges the server's FIN. After this segment is sent, the client enters **TIME_WAIT**.

4.2 TIME_WAIT

After sending the final ACK, the active closer enters **TIME_WAIT** for a duration of $2 \times \text{MSL}$ (Maximum Segment Lifetime). MSL is typically 60–120 seconds, so **TIME_WAIT** lasts 2–4 minutes. Two reasons justify this delay:

1. **The final ACK may be lost.** If the server does not receive the final ACK, it retransmits its FIN. The client must still be present to re-send the ACK. **TIME_WAIT** keeps the client reachable for one more FIN retransmission window.
2. **Prevent 5-tuple reuse confusion.** If a new connection reused the same 5-tuple immediately, stale segments from the old connection still floating in the network could be mistakenly accepted. **TIME_WAIT** ensures those segments have expired (their TTL has elapsed) before the 5-tuple can be reused.³

On a high-connection-rate server (e.g., a load balancer), hundreds or thousands of **TIME_WAIT** sockets are normal. Linux provides `tcp_tw_reuse` to reduce **TIME_WAIT** pressure in specific scenarios.

³Wikipedia contributors. "Transmission Control Protocol — TIME-WAIT." https://en.wikipedia.org/wiki/Transmission_Control_Protocol#TIME-WAIT

4.3 RST: Abrupt Close

A **RST** segment aborts the connection immediately — no graceful teardown, no `TIME_WAIT`. The receiving side discards all buffered data and closes the socket at once. RST is used when:

- A connection request arrives for a port with no listener (“Connection refused”).
- A half-open connection is detected (one side crashed and restarted with no memory of the connection).
- A firewall or middlebox forcibly terminates a connection.

RST carries no data and is never acknowledged.

5 Reliable Delivery in TCP

5.1 Cumulative ACKs and Delayed ACK

TCP uses **cumulative acknowledgment** (inherited from Go-Back-N, as shown in Table 1): the ACK number indicates the next byte the receiver is expecting. A single ACK therefore acknowledges all bytes up to that point. If a sender transmits three segments covering bytes 0–499, 500–999, and 1000–1499, and all arrive, the receiver need only send one ACK with `ack= 1500`.

Delayed ACK (RFC 1122): rather than acknowledging every segment immediately, the receiver waits up to 500 ms *or* until two full-sized segments have arrived, then sends one ACK. This halves the ACK traffic on a stream of full-sized segments and reduces header-to-data ratio on the reverse path.

SACK. When segments arrive out of order, the base cumulative ACK cannot report what has been received beyond the gap. The SACK option adds Selective Repeat-style reporting: the receiver lists the specific byte ranges it holds, and the sender retransmits only the missing ranges. SACK is negotiated at SYN time and is supported by essentially all modern TCP implementations.

SACK in the Wild: What You Will See in Wireshark

SACK is enabled by default on Linux, macOS, and Windows. In practice, *nearly every* TCP connection you capture with Wireshark will use SACK rather than pure cumulative ACKs. You can identify SACK-capable connections by the `SACK_PERM` option in the SYN and SYN-ACK segments, and by `SACK` option blocks in ACKs following any out-of-order delivery.

If you want to observe classic cumulative-ACK (GBN-style) behavior for comparison or demonstration purposes, SACK can be disabled at the OS level:

```
# Linux -- disable SACK system-wide (restore with value=1)
sudo sysctl -w net.ipv4.tcp_sack=0

# macOS
sudo sysctl -w net.inet.tcp.sack=0
```

After disabling, new connections will fall back to cumulative-only ACKs. Note that this affects all connections on the host; restore the default (`=1`) after the demonstration. Alternatively, filter your Wireshark capture to connections whose SYN does *not* carry `SACK_PERM` — older or embedded devices often fall into this category.

5.2 The RTO Problem

Each TCP connection maintains a **Retransmission Timeout (RTO)**. When the RTO fires before an ACK covers the oldest outstanding byte, TCP retransmits. The RTO must be calibrated carefully:

- **Too short:** spurious retransmissions inject unnecessary load, confusing the congestion control machinery and wasting bandwidth.
- **Too long:** recovery from genuine loss is slow; one dropped packet can idle a connection for hundreds of milliseconds.

The right RTO depends on the round-trip time (RTT), which varies continuously with queue depths, routing changes, and path re-routing. A fixed RTO is always wrong; the RTO must be estimated adaptively.

5.3 RTT Estimation: EWMA as a Discrete-Time Low-Pass Filter

5.3.1 The Smoothed RTT Estimator

TCP maintains a running estimate of the RTT using an **Exponentially Weighted Moving Average (EWMA)**:

$$\text{SRTT} \leftarrow (1 - \alpha) \cdot \text{SRTT} + \alpha \cdot R \quad \alpha = \frac{1}{8}$$

where R is the most recent RTT sample. ECE and signals-processing students will recognise this immediately as a **first-order IIR low-pass filter in discrete time**. Expanding the recursion:

$$\text{SRTT}_n = \alpha \sum_{k=0}^n (1 - \alpha)^k R_{n-k}$$

Each past sample is weighted by a geometrically decaying envelope with ratio $(1 - \alpha)$. The *time constant* of the filter — the number of samples before the weight falls to $1/e$ — is approximately $1/\alpha = 8$ samples.

The choice $\alpha = \frac{1}{8}$ is deliberate on two counts:

- **Practical filter response.** Large α (e.g., $\frac{1}{2}$) gives fast response to RTT changes but a noisy, bouncy estimate. Small α (e.g., $\frac{1}{16}$) gives a smooth estimate but responds sluggishly to sudden RTT shifts. $\alpha = \frac{1}{8}$ is a practical compromise.
- **Hardware efficiency.** $\frac{1}{8} = 2^{-3}$: the multiply becomes a right-shift by three bits, important when TCP first ran on low-clock-rate hardware in the 1980s.

Key Result

SRTT is a first-order IIR low-pass filter. Time constant $\approx 1/\alpha$ samples. $\alpha = 1/8 = 2^{-3}$ (right-shift on hardware). Past samples are weighted by an exponentially decaying envelope with ratio $(1 - \alpha) = 7/8$.

5.3.2 Why Mean Alone Is Insufficient: The Role of Variance

Consider two networks, both with mean RTT of 100 ms:

- **Network A:** samples are 95, 98, 102, 100, 101 ms (low variance).
- **Network B:** samples are 40, 180, 60, 200, 20 ms (high variance, same mean).

If we set $RTO = SRTT + 20$ ms: Network A never times out spuriously — every sample is within 7 ms of the mean. Network B fires a spurious retransmit on every spike above the mean + 20 ms, injecting retransmits and congestion signals into an already-variable network. The margin above the mean must *scale with the variability of the RTT*.

Jacobson (1988) identified variance blindness as a root cause of TCP congestion collapse and introduced a second filter to track it:⁴

$$RTTVAR \leftarrow (1 - \beta) \cdot RTTVAR + \beta \cdot |SRTT_{old} - R| \quad \beta = \frac{1}{4}$$

$$RTO = SRTT + 4 \cdot RTTVAR$$

RTTVAR is a second EWMA, this time tracking the *mean absolute deviation* of the RTT around the smoothed estimate — a noise-power estimator for the RTT process. $\beta = 1/4 = 2^{-2}$ (again a right-shift). The coefficient 4 in the RTO formula: for a Gaussian signal, $\pm 4\sigma$ gives 99.994% coverage; RTT distributions are right-skewed (they cannot go negative but can spike high), and the mean absolute deviation slightly underestimates the standard deviation, so 4 provides a practical safety margin. This formula is in Linux TCP today (RFC 6298).

Key Result

$RTO = SRTT + 4 \cdot RTTVAR$. Two-filter adaptive system: SRTT tracks the mean; RTTVAR tracks the variability. RTO is a confidence-interval-style bound that adapts automatically to changing network conditions. Both $\alpha = 1/8$ and $\beta = 1/4$ are right-shifts on hardware.

⁴Jacobson, V. “Congestion avoidance and control.” *SIGCOMM*, 1988.

Example

Numerical walk-through. Current state: $SRTT = 100$ ms, $RTTVAR = 10$ ms. New sample: $R = 150$ ms.

Step 1 — update $RTTVAR$ using the old $SRTT$ (per RFC 6298, use $SRTT$ before updating):

$$RTTVAR \leftarrow \frac{3}{4} \times 10 + \frac{1}{4} \times |100 - 150| = 7.5 + 12.5 = 20.0 \text{ ms}$$

Step 2 — update $SRTT$:

$$SRTT \leftarrow \frac{7}{8} \times 100 + \frac{1}{8} \times 150 = 87.5 + 18.75 = 106.25 \text{ ms}$$

Step 3 — compute new RTO :

$$RTO = 106.25 + 4 \times 20.0 = 186.25 \text{ ms}$$

A single 50 ms RTT spike has widened the RTO from 140 ms (original $100 + 4 \times 10$) to 186 ms, providing margin for future variance without over-reacting. If subsequent samples return to ~ 100 ms, $RTTVAR$ will decay back toward 10 ms and the RTO will tighten again.

5.4 Karn's Algorithm and the Retransmission Ambiguity

When TCP retransmits a segment, an ACK eventually arrives. Which transmission did it acknowledge — the original or the retransmit? There is no way to know. Using an ambiguous sample to update $SRTT$ corrupts the estimate: if the retransmit was fast, $SRTT$ is pulled down spuriously; if it was slow, it is pulled up.

Karn's algorithm: do not update $SRTT$ or $RTTVAR$ from any ACK that follows a retransmission. Only use RTT samples from segments that were sent exactly once.

The **Timestamps option** (RFC 1323) removes the ambiguity entirely. Each segment carries the sender's timestamp; ACKs echo it back. The sender computes the exact RTT for any acknowledged segment regardless of whether it was retransmitted. When timestamps are enabled — the default on Linux and macOS — Karn's algorithm is effectively unnecessary.

5.5 Exponential Backoff

Each consecutive timeout doubles the RTO (up to a maximum of 60–120 s):

$$RTO \leftarrow 2 \times RTO$$

This is **binary exponential backoff** — the same principle used in Ethernet CSMA/CD (Lecture 7) and Aloha.⁵ If the network is dropping packets because it is overloaded, sending faster makes things worse. Backing off reduces injected load and gives the network time to drain its queues. After one successful non-retransmitted segment, the RTO is reset to the $SRTT$ -based estimate.

5.6 Fast Retransmit: Do Not Wait for the Timer

The RTO can be hundreds of milliseconds. A faster loss signal is available through out-of-order delivery. When a receiver gets a segment whose sequence number is higher than expected (a gap

⁵Wikipedia contributors. "Exponential backoff." https://en.wikipedia.org/wiki/Exponential_backoff

exists), it immediately sends an ACK for the *last in-order byte* it has received — a **duplicate ACK**. This duplicate ACK says: “I received something, but I’m still missing an earlier byte.”

TCP’s rule: **three duplicate ACKs** trigger an immediate retransmit of the missing segment, without waiting for the RTO.

Why three? One or two duplicate ACKs can arise from normal IP reordering (packets arriving slightly out of order is common). Three consecutive duplicates are a reliable signal that the segment is genuinely lost rather than merely delayed or reordered.

Worked trace. Six segments are sent; segment 3 is lost:

```

Sender transmits: [seg 1][seg 2][seg 3][seg 4][seg 5][seg 6] -->
                  (X) seg 3 lost in transit
Receiver:
  receives seg 1 --> ACK 2
  receives seg 2 --> ACK 3 (all in order so far)
  receives seg 4 --> dup ACK 3 (1st duplicate: gap detected)
  receives seg 5 --> dup ACK 3 (2nd duplicate)
  receives seg 6 --> dup ACK 3 (3rd duplicate)
Sender: 3 dup ACKs --> retransmit seg 3 immediately (no RTO wait)

```

Fast retransmit substantially improves TCP throughput on paths with occasional single-packet loss, because it avoids the full RTO stall in the common case.

6 Flow Control

6.1 The Problem

The sender and receiver operate at different speeds. The receiving application may not read data from its socket buffer as fast as it arrives. Without a throttle, the buffer fills, new arriving bytes are silently dropped, and the sender’s retransmit logic kicks in — wasting bandwidth and adding latency. Flow control is an explicit, end-to-end mechanism: the receiver continuously tells the sender how much buffer space it has available.

6.2 The Receive Window

Every TCP segment carries a **Window Size** field (rwnd). The effective sender constraint is:

$$\text{bytes in flight} \leq \text{rwnd}$$

where “bytes in flight” = (last byte sent) – (last byte ACKed).

As the application reads data from the socket buffer, the receiver sends ACKs with an increasing rwnd, opening the window. If the buffer fills completely, rwnd= 0 and the sender must stop. TCP handles this via **zero-window probing**: the sender periodically transmits a 1-byte probe segment to check whether the window has reopened.

Example**Flow control trace.**

Receiver has a 16 KB (16384 byte) socket receive buffer. Application is reading slowly.

- Three segments arrive ($3 \times 1460 = 4380$ bytes buffered).
Free space: $16384 - 4380 = 12004$ bytes.
Receiver advertises: **rwnd = 12004**.
- Application reads all 4380 buffered bytes.
Free space: 16384 bytes.
Receiver advertises: **rwnd = 16384**.
- Application stalls; buffer fills completely.
Receiver advertises: **rwnd = 0**.
Sender stops transmitting data; begins zero-window probing.

7 Nagle's Algorithm and Silly Window Syndrome

Two inefficiency pathologies arise at the extremes of small data.

Silly Window Syndrome. Suppose the receiver opens its window by only 1–2 bytes because the application reads very slowly. The sender dutifully fills this tiny window — sending a full 20-byte TCP header plus 20-byte IP header to carry 1 or 2 bytes of payload. The solution is **Clark's algorithm**: the receiver does not advertise a window smaller than $\min(\text{MSS}, \frac{1}{2} \text{ buffer})$ until it can offer a “reasonable” window.⁶

Nagle's Algorithm. An application making many small writes (e.g., interactive SSH keystrokes sending one character per segment) generates a large number of tiny segments. Nagle's rule: if there are bytes outstanding in the network (unacknowledged), buffer additional small writes until an ACK arrives, then transmit the accumulated data as a single larger segment.⁷ The trade-off: Nagle reduces small-packet flooding and improves efficiency, but it adds latency proportional to the RTT on each small write. Applications requiring low latency (VoIP, gaming clients, interactive shells) disable it with the `TCP_NODELAY` socket option.

8 TCP State Machine

TCP connections move through a well-defined set of states. Table 3 lists the states relevant to this course.

⁶Wikipedia contributors. “Silly window syndrome.” https://en.wikipedia.org/wiki/Silly_window_syndrome

⁷Wikipedia contributors. “Nagle's algorithm.” https://en.wikipedia.org/wiki/Nagle%27s_algorithm

Table 3: Key TCP states

State	Who enters it	Meaning
LISTEN	Server	Waiting for an incoming SYN
SYN_SENT	Client	SYN sent; waiting for SYN-ACK
SYN_RECEIVED	Server	SYN-ACK sent; waiting for final ACK
ESTABLISHED	Both	Data transfer in progress
FIN_WAIT_1	Active closer	FIN sent; waiting for ACK
FIN_WAIT_2	Active closer	FIN ACKed; waiting for server's FIN
CLOSE_WAIT	Passive closer	FIN received and ACKed; waiting for app to close
LAST_ACK	Passive closer	FIN sent; waiting for the final ACK
TIME_WAIT	Active closer	Final ACK sent; waiting $2 \times \text{MSL}$
CLOSED	Both	No connection state exists

Demo

```
# Show sockets in all states (including LISTEN and TIME_WAIT)
ss -t -a

# Watch TIME_WAIT entries appear after closing connections
watch -n 1 'ss -t -a | grep TIME-WAIT | wc -l'
```

TIME_WAIT entries are visible after teardown. On a busy server, hundreds or thousands of TIME_WAIT sockets are normal.

Discussion Questions

1. The TCP header has no explicit “payload length” field. How does a receiver determine where the data portion of a segment ends?
2. Why does the three-way handshake need to be three messages rather than two? What would go wrong with a two-way handshake if the server’s SYN-ACK were lost?
3. After a connection closes, a new connection between the same pair of endpoints uses the same port numbers. Why does ISN randomization protect against stale segments from the old connection?
4. A TCP sender has 8 KB outstanding (unacknowledged). The most recent ACK advertises $\text{rwnd} = 4 \text{ KB}$. How many additional bytes can the sender transmit?
5. Why does fast retransmit require three duplicate ACKs rather than one?
6. A receiver’s application reads data very slowly and the socket buffer is nearly full. What value will the receiver advertise in the Window Size field, and what will the sender do in response?
7. The SRTT filter has $\alpha = 1/8$ and the RTTVAR filter has $\beta = 1/4$. RTTVAR responds more quickly than SRTT does. Why is it desirable for the variance estimator to be more agile than the mean estimator?

References

1. Wikipedia contributors. “Transmission Control Protocol.” Wikipedia.
https://en.wikipedia.org/wiki/Transmission_Control_Protocol
2. Wikipedia contributors. “Maximum segment size.” Wikipedia.
https://en.wikipedia.org/wiki/Maximum_segment_size
3. Wikipedia contributors. “Nagle’s algorithm.” Wikipedia.
https://en.wikipedia.org/wiki/Nagle%27s_algorithm
4. Wikipedia contributors. “Silly window syndrome.” Wikipedia.
https://en.wikipedia.org/wiki/Silly_window_syndrome
5. Wikipedia contributors. “TCP congestion control.” Wikipedia.
https://en.wikipedia.org/wiki/TCP_congestion_control
6. Wikipedia contributors. “Exponential backoff.” Wikipedia.
https://en.wikipedia.org/wiki/Exponential_backoff
7. Wikipedia contributors. “Transmission Control Protocol — TIME-WAIT.” Wikipedia.
https://en.wikipedia.org/wiki/Transmission_Control_Protocol#TIME-WAIT